5. Bitwise operators (6)

- Bitwise operators perform on the binary representations of integers. **<u>OR</u>** Bitwise operators work on individual bits of integers. These are basically used for testing or shifting bits.

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12, which is 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) will give 61, which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49, which is 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61, which is 1100 0011 in 2's complement form. Result in decimal : ~A = -(A+1)
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 0000 1111

```c
int main()
{
    int a=5, b=3;
    printf("(a & b) = %d \n",(a&b)); //and
    printf("(a | b) = %d \n",(a|b)); // or
    printf("(a ^ b) = %d \n",(a^b)); // xor
    printf("(~a) = %d \n",(~a)); // bitwise negative OR Complement operator
    printf("(~b) = %d \n",(~b)); //bitwise negative
    printf("(a << 1) = %d \n",(a << 1)); // left shift operator
    printf("(a >> 2) = %d \n",(a >> 2)); // right shift operator
    getch(); // optional
}
```

C:\Users\hira\Desktop\hello.exe

```
(a & b) = 1
(a | b) = 7
(a ^ b) = 6
(~a) = -6
(~b) = -4
(a << 1) = 10
(a >> 2) = 1
```

Not Confuse

<table>
<tr><th>Bitwise</th><th>Logical</th></tr>
<tr><td>a & b</td><td>a && b</td></tr>
<tr><td>a | b</td><td>a || b</td></tr>
<tr><td>a ^ b</td><td>a != b</td></tr>
<tr><td>~a</td><td>!a</td></tr>
</table>

6. Assignment Operators (11)

- Assignment operators are used to assign a value (or) an expression
  (or) a value of a variable to another variable.
- Syntax : variable name=expression (or) value (or) variable
  - ✓ Ex : x=10;
    y=a+b;
    z=p

▪ i += 2 (The operator += is called an assignment operator.)

Operator	Example	Meaning
+=	x += y	x=x+y
-=	x -= y	x=x-y
*=	x *= y	x=x*y
/=	x /= y	x=x/y
%=	x %= y	X=x%y

<table>
<tr><td>=</td><td>Assignment operator</td></tr>
<tr><td>+= -=</td><td>Addition/subtraction assignment</td></tr>
<tr><td>*= /=</td><td>Multiplication/division assignment</td></tr>
<tr><td>%= &=</td><td>Modulus and bitwise assignment</td></tr>
<tr><td>^= |=</td><td>Bitwise exclusive/inclusive OR assignment</td></tr>
<tr><td><<= >>=</td><td></td></tr>
</table>

If x *= y + 1 means → x = x * (y + 1) rather than x = x * y + 1

```c
int main()
{
    int a = 10;
    int b =a;
    printf("a = %d \n",a);
    printf("b = %d \n",b);
    b*=a+b;
    printf("value of b = %d",b);
    getch(); // optional
    return 0;

}
```

C:\Users\hira\Desktop\hello.exe

```
a = 10
b = 10
value of b = 200
```

7. Ternary or Conditional Operator (1)

- In c, the ternary operator is a shorthand way to write simple if-else statement, It's the only operator in c that takes three operands.
- It is also called Conditional Operator.
- Syntax : Condition ? expression_if_true : expression_if_false;
- Example: int a=10,b=20;
  - Int max = (a>b) ? a : b ;

Explain this line

int max = (a>b) ? a : b ;

```
int max;
if(a>b)
{
  max =a;
}
else
{
  max =b;
}
```

```c
#include<stdio.h> // printf() scanf()
#include<conio.h> // getch()
#include<stdbool.h> // bool
int main()
{
    int a= 10, b=20;
    int max_value = (a>b) ? a: b;
    printf("Greatest Value = %d",max_value);
    getch();
     return 0;
}
```

```
C:\Users\hira\Desktop\hello.exe
Greatest Value = 20
```